Knowing What You Need Is Not Enough: The Policy Bottleneck in Homeostatic Agents

Anonymous Author(s)
Anonymous Institution
anonymous@email.com

Abstract

A homeostatic agent acts to keep internal drives near their setpoints, so its behaviour hinges on the *satiation signal* g: how much each outcome quiets each drive. Hand-designing g risks an agent that regulates flawlessly toward the wrong thing — the Satiation Signal Problem. We ask whether g can instead be learned from observable consequences, in a domain anyone can picture: a student with forty study sessions before an exam, choosing how hard to push, pulled between wanting to progress and wanting to feel competent. Across a dose–response sweep of environmental adversity, with predictions hashed before execution and paired analysis over 100 student profiles, we decompose the gap between the homeostatic agent and a simple state-estimating baseline. Under the agent's hand-chosen action policy, signal quality is the smallest term: a *perfect* g buys 7.7% of the gap while re-tuning six policy coefficients buys 25.0%. But the two are complementary, not competing. Searching the policy harder roughly doubles what a good signal is worth — $+0.047 \rightarrow +0.121$ across five independent searches — so a satiation signal only pays once the channel can carry it. Over half the gap survives both interventions. Knowing what satisfies the agent is necessary and nowhere near sufficient: the channel that turns satiation into action is what binds. Homeostatic regulation still earns a role, but narrower than we first claimed: it buys viability rather than optimality, and only against baselines lacking any rest mechanism. Diagnosing why sent us back to our own applicability framework, which we find under-specified — and we propose a sixth condition, suggested by a post-hoc test, under which the ordering reverses.

Code and data to reproduce every number, table and figure:
`https://anonymous.4open.science/r/gamma-n2-policy-bottleneck-0000`
(anonymized for review; replace with the archival link on acceptance)

1 Introduction

What moves an agent to act? In biology, action is not a default state — it emerges when internal variables leave viable ranges [1]. Yet dominant paradigms in autonomous agent design optimize external objectives and rarely treat internal regulatory integrity as a first-class concern. What the agent itself *needs* is systematically absent.

That absence shows in where engineering effort has gone: from prompt engineering to context engineering to harness and loop engineering, successive scaffolding layers around a capable model. Each adds capability; none adds motivation. This line of work bets on a different move — from **agent engineering** to **agent cybernetics**: less effort adding capabilities, more inserting the regulatory structure that decides when and why a capability should be exercised. It matters because deployment surfaces the failure modes capability alone does not address — agents that over-optimize, accumulate

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

context until collapse, or pursue goals long after they stop being appropriate. Autonomy is the capacity to *remain viable while doing*, not just to *do*.

Prior work in this line established feasibility. The Pro-Action Operator Γ showed that coupled homeostatic drives integrated with LLM reasoning produce behaviour differentiated by opponent type in iterated social dilemmas (ICML 2026, anonymized). A single-drive variant produced emergent regulatory behaviour in debate facilitation without any reinforcement learning — and surfaced a structural limitation: g was hand-designed, and when misaligned the system regulated toward the wrong attractor (SANLP 2026, anonymized). Tied to a structural metric it regulated participation but not semantic fidelity; tied to an LLM judge, context collapse *quintupled*. The loop worked — toward the wrong thing.

We call this the **Satiation Signal Problem (SSP)**: how is g defined so the system regulates toward a desired external goal? It is one problem inside the broader agenda of agent cybernetics, not the agenda itself — but it gates the rest, because the mechanism is value-neutral and the signal feeding it decides what it regulates toward. So we ask: **can a homeostatic agent learn g from the observable consequences of its own actions, and use it to regulate toward an externally useful goal?** The domain is a student with forty study sessions before an exam, choosing difficulty guided by two drives — one pushing, one braking — while never observing its own ability, fatigue or frustration.

A single operating point cannot answer this: an earlier iteration produced a null admitting two incompatible readings — the signal does not matter, or the environment never applied pressure enough to make it matter. We therefore sweep the pressure, hash the predictions before execution, gate every sweep point, and analyze 100 profiles as the paired design it is.

Our central result is a decomposition rather than a verdict. Holding the agent's hand-chosen policy fixed, a perfect oracle g buys 7.7% of the gap to a state-estimating baseline while re-tuning six coefficients buys 25.0%, and most of the gap survives both. But the two terms are not independent, and that turns out to be the finding. Search the policy harder and the value of a good signal roughly doubles, from $+0.047$ to $+0.121$ (p=.022 over five independent searches): a satiation signal is worth little until the channel can carry it. Knowing what you need is necessary and nowhere near sufficient. Where regulation earns a role is viability, and there too we narrow an earlier claim: with rest disabled the homeostatic agent still reaches 93.5% dropout against the estimator's 100%, so viability is bought by having *some* rest mechanism.

Four contributions follow: the decomposition, with paired effect sizes and confidence intervals and replicated on the goal metric; the matched-tuning result, which shows a *learned* signal matches a constant under a modest search budget and beats it only under a larger one; a methodology — dose–response sweeps with per-point gating, hashed predictions, multiplicity-corrected existence tests, and multi-seed tuning with held-out validation — for architectures making claims about *regimes*; and a revision of our own applicability framework, including a sixth condition under which the ordering reverses. We also report, in the body rather than a footnote, three of our own claims that stricter protocols forced us to withdraw.

2 Background and Related Work

In game AI, Zubek [8] synthesized the needs-based approach exemplified by The Sims — drawing on robotics and his own industry practice rather than on that title's implementation — in which agents act to satisfy a vector of decaying needs. It produces believable behaviour without scripting, but the mappings are hand-authored: the agent never discovers what satisfies it; it is told. What if it had to learn them?

That question leads to biology. Cannon [1] formalized homeostasis as maintaining internal variables within viable ranges through negative feedback — an architecture robust enough to underpin temperature, glucose, and fluid regulation across all vertebrates, but one that corrects *after* deviation is detected. Sterling and Eyer [2] extended this with allostasis: predictive adjustment based on anticipated demand. An allostatic system does not wait for glucose to drop. The distinction is temporal, and it predicts a failure mode our agent encounters.

Keramati and Gutkin [3] bridged these principles with RL, defining primary reward as the reduction in homeostatic deviation — proportional to $D(H_t) - D(H_t + K_t)$, the drive before and after an outcome — and proving that maximizing discounted reward is equivalent to minimizing discounted

deviation, so that reward-seeking *is* stability-seeking. Yet the drive function and its parameters are fixed: the agent learns *which actions* reduce deviation, not *how much each outcome satisfies each drive*. That mapping is prescribed. Our work picks up exactly there.

The gap matters beyond HRL. The successor representation literature [5–7] showed that agents benefit from modelling how the environment affects future states — which our results identify as the necessary next step. Ashby's Law of Requisite Variety [4] constrains what our agent can achieve, and our residual term is best read as a variety failure located not in the drives but in the channel connecting them to action. A further critique, the Rule-Relocation Problem (various authors, 2025–2026), notes that the setpoint is itself a designer-imposed norm: normativity is relocated from reward function to setpoint rather than eliminated. The SSP is a special case, g being the relocated norm.

3 When Is a Problem Regulatory?

A framework that cannot say where it does *not* apply is not a framework. We hold that a problem is regulatory when it exhibits five properties:

1. **Variables that must be sustained within viable ranges** — not maximized, but kept in a band over time. Sleep is the everyday case: neither zero nor sixteen hours beats seven, and the cost of leaving the band compounds.
2. **Antagonistic tensions that must be held without collapse** — satisfying one need destabilizes another, and the trade-off cannot be resolved once and forgotten. A household budget balances spending against saving continuously; training balances stimulus against recovery.
3. **Non-linear dynamics** — crossing a threshold has consequences that do not smoothly extrapolate. Overtraining does not degrade performance gradually; it produces an injury.
4. **Situated information particular to the agent** — the agent observes its own condition in ways an external orchestrator cannot. A coach sees your lifts; only you feel the joint about to give.
5. **An objective learning signal** — outcomes are measurable without human judgment or circularity.

These five are our starting position; Section 7.4 revises them in light of what the experiment showed, including a sixth we did not anticipate. The self-regulated learner satisfies (5) unambiguously: an answer is correct or it is not. Conditions (1)–(4) are what our experiment manipulates, and a central methodological finding here is that they must be manipulated rather than assumed. The everyday version: push to your maximum every day from the start and you quit; never push and you stagnate. The right behaviour is not a schedule but a dynamic balance between the drive to improve and the need to recover, adjusted from how your body responds.

A note on the target band. We measure the difficulty band that maximizes expected learning gain in the simulator. This is inspired by, but narrower than, Vygotsky's Zone of Proximal Development [9], which concerns the gap between independent and assisted performance. Our environment models no scaffolding, so we call the measured quantity the **optimal-challenge band** and use time-in-band (TIB) as its metric, reserving the ZPD label for the motivating construct.

4 Architecture

4.1 Two drives, and what they feel like

Everything below follows from one design choice: the agent is given *needs*, not goals — two of them, both deficits that grow until something quiets them.

The first, δ_{prog}, is the itch of coasting — what you feel in the third week of lifting the same comfortable weight: nothing is going wrong, and that is exactly the problem. It grows as sessions pass, faster the more you are already succeeding, and is quieted only by genuine progress. Left alone it pushes upward.

The second, δ_{conf}, is the flinch after a bad run — what makes you re-rack the bar after two failed lifts. It jumps on errors and is quieted by easy successes. Left alone it pulls downward.

Neither drive is a goal, and neither knows anything about difficulty levels. The agent never observes its ability, fatigue or frustration — only which level it chose, whether it was right, and how long it took. Behaviour is whatever falls out of two deficits pulling opposite ways.

4.2 Why two drives and not one

A single drive cannot tell the two failure modes apart: if the agent is dissatisfied, is the material too easy or too hard? A lone drive registers only "not satisfied" either way. This is Ashby's requisite variety [4] in concrete form, and two antagonistic drives are the minimum.

Formally, each drive i has a setpoint θ and evolves per session as

$$\delta_{i,t+1} = \delta_{i,t} + \lambda_i m_i - \alpha_i \, g_i[k] \, \mathbf{1}[\text{satiating outcome}] + w_{ij} \, p_i(\delta_i, \theta) \, \mathbf{1}[\delta_j > \theta + \epsilon] \tag{1}$$

reading left to right: the drive grows by a basal λ_i scaled by observable mastery m_i (hunger between meals); it shrinks by $\alpha_i g_i[k]$ when a satisfying outcome arrives at level k; and the last term couples the drives, active only when the other is above setpoint. That coupling is modulated by the *receiving* drive's own pressure, $p(\delta, \theta) = \min(1, |\delta - \theta|/0.8)$, so a drive at setpoint ignores its counterpart while one already under strain responds sharply — the reason a warning lands differently depending on how close to your limit you are. Pressure is symmetric because an under-challenged learner is not content, they are bored, and boredom generates its own pressure to act.

4.3 The satiation signal, and why it is the crux

The term $g_i[k]$ is where the design problem lives: it decides how much a correct answer at level k counts as progress. Set it wrong and the machinery still works perfectly — it just regulates toward the wrong place. Fix g to a constant and every correct answer, however trivial, quiets the progress drive equally; the agent settles at the easiest level it can find and calls itself satisfied. That is the SSP in one sentence.

The learning rule uses only what the agent can see — a running accuracy per level, $\text{acc}[k]$:

$$g_{\text{prog}}[k] \leftarrow (1-\eta)\, g_{\text{prog}}[k] + \eta \cdot 4\, \text{acc}[k]\,(1-\text{acc}[k]), \qquad g_{\text{conf}}[k] \leftarrow (1-\eta)\, g_{\text{conf}}[k] + \eta \cdot \text{acc}[k] \tag{2}$$

The intuition is ordinary: a level where you get everything right teaches little, and so does one where you get everything wrong. A mastered level yields low g_{prog}, correct answers barely satisfy the progress drive, it keeps growing, and the agent climbs. An impossible level also yields low g_{prog} while errors inflate confidence, and the agent retreats. In between, satiation balances growth. **The target band is never encoded — it is where the dynamics come to rest.**

One property bounds everything that follows: $4a(1-a)$ is symmetric with its maximum at $a{=}0.5$, whereas expected gain in our environment is maximized at a difficulty gap of 0.40, where accuracy is only 0.354. The rule's peak sits on the wrong side of the true optimum, capping achievable fidelity regardless of how much data the agent collects.

4.4 From drives to actions, and what that costs

Something must convert two numbers into one of four choices: go up, stay, go down, or rest. We use four linear logits over the drive deviations d_p and d_c with six coefficients, sampled by softmax, with REST gated by normalized frustration; ablation arms replace sampling with argmax, remove the rest gate, or re-tune the coefficients (Appendix B). We flag this stage because it, not the satiation signal, is where the architecture loses — and its six coefficients were chosen by hand, which we treat as a variable to measure rather than a detail to defend.

The operator itself is deliberately cheap: per session, $O(n^2)$ coupling updates, $O(A)$ logits and $O(1)$ signal updates — about twenty floating-point operations at $n{=}2$, $A{=}4$, $K{=}5$ — with $O(n^2 + K)$ state, no gradient, no value function and no replay. Measured cost is $14\ \mu$s per decision.

5 Experimental Design

5.1 The environment, and the two things that make it a test

The simulated student has a hidden ability h; correctness at level k follows a Rasch model [10], $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning peaks at a moderate stretch above current ability and falls off both ways, and failing at something challenging still teaches, so playing safe is not trivially optimal. Effort accumulates and drags down both effective ability and the capacity to learn; recovery slows sharply past a threshold, so exhaustion cannot simply be slept off. Resting costs one of the forty sessions and induces mild forgetting. Frustration rises with errors, rises further when failing at something previously mastered, falls with easy successes, and if sustained near a personal threshold for three sessions the student quits for good. Fatigue also carries a deferred cost *during* the episode: it degrades both effective ability and the capacity to learn at every session, so strain that is cheap now is expensive later.

Primary outcome. Every contrast in this paper is on **learning gain**, $h_{\text{final}} - h_0$: *latent* ability accumulated over the budget. Terminal fatigue is therefore not penalized in the primary outcome directly — it acts through the trajectory, by suppressing how much is learned in each fatigued session. A terminal exam score on *effective* ability is reported as a secondary diagnostic only (Appendix 15); it is not dropout-adjusted, so arms that quit still receive a score, and it should not be read as the task objective. The non-bankable alternative, mean readiness, is analyzed separately in Section 7.4.

Two details are load-bearing. Fatigue must carry a *deferred* cost, because an estimator updating from outcomes already tracks current fatigue for free — an accumulator earns its keep only if strain that is cheap now is expensive later. And quitting must be reachable, or viability is not a live variable. An earlier version of this environment failed both, which is why it produced an uninterpretable null.

5.2 Turning the adversity up, rather than picking one setting

The primary axis is ϕ, the severity of non-linear fatigue, over $\{0, 0.3, 0.6, 0.9, 1.2, 1.6\}$. Turning it up makes exhaustion bite, which produces errors, then frustration, then reachable quitting — one knob activates both the non-linearity and the viability pressure. A second axis moves the *peak* of the learning curve mid-episode, leaving any agent that models the curve misspecified in a way re-estimation cannot repair (Section 6.6).

Sweeping raises its own problem: at extreme adversity an environment can stop discriminating between good and bad policies. We therefore re-run a quality gate at *every* point — hiding at level 1 must stay unprofitable, the quitting boundary must bite, an oracle policy must beat random. It passes for $\phi \leq 1.2$ and fails at $\phi \geq 1.6$, where severe fatigue makes hiding viable and the trap dissolves. The gate thresholds were not in the committed text, so **the gated range is an exploratory analytic choice, not part of the committed test**. We report the crossover with the excluded point included (Appendix C), where it is larger; P2 and P6 are not re-derived under inclusion and are conditional on the gated range.

5.3 Committing to predictions before looking

A sweep with ten arms and six points offers many places to find a story after the fact. The predictions below were therefore written as plain text and hashed with SHA-256 before any sweep executed:

`6e888c7e46e1dd306b6adc601a45cc08ff914a24d2f474a913a1ff2907bfc084`

Anyone can recompute the digest from `predictions.txt` in the supplement. It is self-attested, not a third-party registry: it makes post-hoc editing detectable, nothing more. The gate thresholds were *not* in the committed text.

- **P1 (viability).** Estimation without viability management collapses as ϕ rises.
- **P2 (robustness).** Γ arms degrade less than the estimator across the sweep.
- **P3 (crossover).** Some Γ arm overtakes the estimator within the valid range.
- **P4 (policy).** Argmax beats softmax throughout.

- **P5 (signal).** If g's quality matters behaviourally, oracle separates from a fixed constant.
- **P6 (learnability).** g validity decreases as adversity rises.
- **P7 (misspecification).** Under a change in the *shape* of the learning curve, Γ arms gain relative ground against the now-misspecified estimator.

5.4 Arms and how they are compared

One caveat applies to every arm: the rest mechanism — Γ's strain gate and the baselines' rule alike — normalizes frustration by the profile's private threshold, which a real learner would not disclose. It is a shared rather than differential advantage, but it makes viability easier than a setting where frustration must be estimated; claims about unobserved state here refer to ability and fatigue, not frustration. Each arm isolates one suspect. Γ`_oracle`, Γ`_fixed` and Γ`_naive` vary only the satiation signal; Γ`_argmax` and Γ`_restdrive` vary only the policy; `rule` and `estimator` are non-homeostatic baselines, each run with and without its rest mechanism so viability separates from level selection. The estimator is a ten-line agent that tracks ability from outcomes and picks the level its model says is best.

100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points = 60,000 episodes. Every profile faces every arm, so contrasts use **paired t-tests on profile-level means** (n=100) with 95% CIs and d_z — an earlier analysis of ours used independent-samples tests here and reported a null that pairing overturns. Two precautions follow. Because several conclusions rest on the *absence* of an effect, we run TOST equivalence tests against δ=0.05, with sensitivity across $\delta \in [0.02, 0.10]$. And because P3 is an *existence* claim over arms and adversity points, we evaluate it with a max-statistic sign-flip permutation test over all arm-by-ϕ cells rather than quoting the winning one.

6 Results

6.1 Main sweep

Across the valid range the estimator falls from $+1.132$ to $+0.156$ while Γ_learned moves only from $+0.248$ to $+0.143$; dropout stays at zero for every arm that can rest and climbs from 16% to 100% for the estimator that cannot (Appendix D).

P1 confirmed. `estimator_norest` rises from 16% to 100% dropout: estimating your state perfectly is worthless if you quit in session 20. **P2 confirmed**, and tested as an interaction rather than a ratio of aggregates: fitting each profile's own gain-versus-ϕ slope, the estimator falls at -0.808 per unit ϕ while Γ_learned falls at -0.070, a paired difference of $+0.737$, CI $[+0.706, +0.769]$, d_z=$+4.65$. The regulated agent is flatter, not merely lower.

6.2 Decomposing the gap: signal, policy, residual

All figures in this subsection are for ϕ=0 and do not transfer across the sweep, where the gap itself shrinks sharply (Appendix D). At ϕ=0 the estimator leads by $+0.846$ on the 50 profiles never used for any tuning. We decompose that gap with a 2×2 over signal quality (constant vs. oracle g) and policy parameterization (hand-chosen vs. coefficients re-tuned by a 250-candidate search; Appendix L). The components sum exactly (Appendix E): signal $+0.065$ (7.7%), policy $+0.211$ (25.0%), interaction -0.018 (-2.1%), residual $+0.588$ (69.5%). Against the model-free estimator of Section 6.4, arguably the fairer comparator, the gap is $+0.576$ and the shares become 11.2, 36.6, -3.1 and 55.2%: the ordering is unchanged, only the magnitudes move.

Is the oracle really an oracle? The progress drive is satiated only on correct trials, so its expected reduction is $p \cdot \alpha \cdot g_{\text{prog}}$: dividing expected gain by a constant makes the drive chase $p \cdot \mathbb{E}[\text{gain}]$, not $\mathbb{E}[\text{gain}]$. Re-running with $g_{\text{prog}} = \mathbb{E}[\text{gain}]/p$ moves the arm from $+0.335$ to $+0.343$ and the signal share from 7.7% to 8.6%: the misalignment is real but immaterial, since with five discrete levels both definitions share an argmax. We nonetheless call this an **expected-gain oracle** rather than a perfect g — it is ground truth for g_{prog} only, with a heuristic g_{conf}.

We stress what the residual is and is not. It is the portion unexplained by *the two interventions we ran* — one signal substitution and one six-coefficient search. It also absorbs unsearched policy space, the

fixed action representation, and any joint signal–policy optimum those searches missed. It is not a proven structural constant.

6.3 Matched tuning: how much a signal is worth depends on the policy

The shared coefficients above were optimized against stable signals, so applying them to a noisy learned signal is not a fair test — a point we owe to review. We therefore repeated the search per signal. A first version used one search each and validated on the profiles it had tuned on; it produced a non-monotonic ordering in which a learned g beat a perfect one, which we initially reported as the headline. It does not survive a stricter protocol: independent searches per signal, validated on **50 profiles never used for tuning**.

At the smaller budget the picture is the one we first reported: the oracle beats a constant in five of five searches, by $+0.047$ on paired held-out profiles (CI $[+0.036, +0.058]$, d_z=+1.22), while a *learned* g is **statistically equivalent** to a constant — $+0.008$, CI $[-0.002, +0.018]$, p=.11, TOST against δ=0.05 giving $p < 10^{-4}$.

That equivalence is budget-dependent. At 1200 candidates, with the same five search seeds, the oracle's advantage over a constant grows to $+0.121$ (CI $[+0.029, +0.213]$, p=.022 across searches, $5/5$ wins), while the learned signal's advantage grows to $+0.094$ but is *not* statistically established once uncertainty over the search itself is accounted for (CI $[-0.008, +0.195]$, p=.062, though also $5/5$ in direction). Searching the policy harder did not make the signal irrelevant; it made the signal matter roughly $2.6\times$ more.

An earlier three-seed version showed a larger effect and a drop in the constant-g cell that we attributed to overfitting; with five seeds the constant recovers to $+0.487$ and the gap shrinks. That drop was search noise, and we report the corrected figures.

The honest conclusion is about complementarity, not ranking. **A satiation signal is worth little until the channel can carry it**. Two earlier claims of ours fall with this: that a learned g buys nothing over a constant holds only at the smaller budget, and even at the larger one its advantage is directional rather than significant; and that signal quality is simply the smallest term holds under a fixed policy but misleads as a general statement. What survives, and what the title asserts, is that knowing what you need is necessary and far from sufficient — over half the gap outlives both interventions at either budget.

6.4 Is the estimator winning because it knows the objective?

The estimator does more than track ability: it also knows which level maximizes learning gain — privileged knowledge of the simulator's objective, which might explain the gap by itself. We therefore added an estimator that keeps ability tracking but discards that model, matching the level to its ability estimate. Removing it costs the estimator in the benign regime ($+1.132 \rightarrow +0.851$) but *helps* it under adversity, where the model is misspecified ($+0.156 \rightarrow +0.275$ at ϕ=1.2; Appendix H).

So the answer is no: stripped of its objective model the estimator stays three to six times better than Γ. But this baseline costs us a claim. **The crossover does not survive against it** — at ϕ=1.2 the model-free estimator scores $+0.275$ against Γ_argmax's $+0.201$. We therefore withdraw the crossover as evidence that homeostatic regulation overtakes state estimation, and retain it only as evidence that a *misspecified* objective model degrades faster than drive-based regulation.

6.5 Signal quality across adversity, and the crossover

P5 is confirmed, with a boundary (Appendix F). At ϕ=0 the expected-gain oracle is worth $+0.063$ (d_z=+1.59, p=6×10^{-29}); from ϕ=0.3 onward the difference falls inside the equivalence bound at every margin from 0.02 to 0.10 — practical equivalence, not a proven null. This corrects an earlier analysis of ours that applied independent-samples tests to this paired design and reported no separation anywhere: a Type II error from discarding the within-subject structure.

P4 confirmed, with the effect growing monotonically from d_z=+0.21 at ϕ=0 to d_z=+3.23 at ϕ=1.2 (full contrasts in Appendix G).

P3 confirmed against the model-based estimator — though not against the stronger one of Section 6.4. At ϕ=1.2 the regulated agent overtakes by $+0.045$, CI $[+0.028, +0.062]$, and since P3 is an existence claim a max-statistic sign-flip permutation over all Γ arms and valid ϕ gives p=.0005 (max t=+5.18 at that cell). Two qualifications: the crossover belongs to the Γ_argmax ablation, not the main agent (Γ_learned scores $+0.143$ against $+0.156$), and $+0.045$ sits below our own δ=0.05 threshold — a robust reversal of ordering, not a large one. Excluding ϕ=1.6 was conservative: including it the contrast grows to $+0.089$, d_z=+1.54.

The argmax contrast is not a clean isolation of sampling noise: changing the action rule moves the entire closed loop — visitation, fatigue, rest frequency, and the resulting g estimates. Γ_argmax attains higher signal validity ($+0.550$ vs. $+0.493$ at ϕ=0) under an identical learning rule, because systematic visitation yields cleaner estimates. Read P4 as "the deterministic policy is better," not as "sampling noise costs this much."

P6 confirmed but policy-dependent. Validity for Γ_learned falls from $+0.493$ to $+0.281$; for Γ_argmax it holds at $+0.452$. Learnability is therefore not bounded by non-stationarity as such — it is bounded by non-stationarity *under a stochastic policy*.

6.6 Model misspecification, not situated information

Moving the learning curve's peak mid-episode leaves the estimator's model wrong in a way re-estimation cannot fix. **P7 is refuted**: the gap does not shrink but *widens*, from -0.181 to -0.198 (d_z -1.02 to -1.26, all $p < 10^{-16}$; Appendix M) — at the largest shift the estimator picks a level yielding roughly 13% of achievable gain and still wins threefold. This does *not* refute situated information, and we were too quick to say it did: the manipulation corrupts the competitor's model rather than granting Γ a private signal, and Γ's own rule is anchored at a=0.5, so when the peak moves the estimator's fixed assumption ends up *closer* to truth than Γ's. Condition (4) therefore **remains untested**.

7 Discussion

7.1 Did we answer the question we asked?

Can a homeostatic agent learn g from observable consequences? Yes — validity $+0.493$ against a $+0.994$ ceiling. *Can it use that signal to regulate toward the external goal?* Only marginally: a perfect signal buys under a tenth of the distance to a simple estimator while a rigid policy buys nearly four times more, and a *learned* signal is indistinguishable from a well-chosen constant. The satiation signal is neither the bottleneck nor irrelevant; it is the smallest of the terms.

7.2 Is the homeostatic mechanism buying viability?

Our most confident claim, and it did not survive. The comparison behind it pitted the regulated agent against an estimator with *no* rest mechanism — only one side could rest. Disable rest on both and the estimator hits 100% dropout, Γ 93.5%: the predicted direction, not a difference in kind, and arguably a foregone conclusion in an environment where fatigue accumulates without bound. The informative comparison is the one where both can rest — and there both sit near zero dropout, with the estimator also ahead on gain. Viability is bought by having *some* rest mechanism; a hand-coded rule recovers nearly all of it.

We also claimed regulation folds stopping into the machinery that picks difficulty. Appendix B says otherwise: the default agent's rest logit is frustration-gated, so the drives barely participate. The claim holds only for `restdrive` — which pays 4–17$\%$ dropout for it.

7.3 Our framework says this is a regulatory problem. Why did regulation lose?

Section 3 lists five conditions under which we expect regulation to pay. Does this domain satisfy them? If it does, the framework is in trouble.

Conditions (3) and (5) hold cleanly, and (3) delivers the predicted signature: across the sweep the estimator loses 86% of its gain, Γ only 42%. Condition (1) holds *cheaply* — quitting is reachable,

but a one-line guard erases it. Condition (2) holds nominally: the estimator has no confidence drive and still wins, so the antagonism is not irreducible.

Condition (4) fails outright, and we can price it. We ran the estimator's own update rule as an external observer — it sees only the level attempted and whether the answer was correct, never ability, fatigue or frustration — and measured $|\hat{h} - h_{\text{eff}}|$ from session 10 onward across 40 profiles $\times$ 5 seeds. Mean absolute error is 0.32 at ϕ=0 and 0.86 at ϕ=1.2, on a scale where difficulty levels sit 1.0 apart. In the benign regime nothing is private.

That is the crux. Conditions (1), (2), (3) and (5) say regulation is *needed*; only (4) says the agent must be the one doing it. It is load-bearing, and we treated all five as equals. But note what the measurement shows: that *this* environment grants no private information about ability, not that Γ would exploit such information if it had any — no arm is ever granted a signal the estimator lacks. Condition (4) is diagnosed as absent here and remains **untested as a mechanism**. Suggestively, Γ gains ground exactly where reconstruction becomes hard: as the error goes from 0.32 to 0.86, the estimator collapses and Γ does not.

7.4 Is a sixth condition missing?

We think so, and it explains our result better than the other five. Homeostasis is a *maintenance* formalism: no horizon, no notion that an exam falls on session 40. Our task has a terminal objective, and accumulated gain can be banked, so more is always better. We were asking a regulator to maximize a payoff. Hence a sixth condition: **the objective must be the maintenance itself, not a terminal payoff that maintenance merely enables.**

What would we expect if that is right? Replace the terminal objective with one that *cannot* be banked — mean readiness, the score you would get if the exam were today, averaged over every session — and the ordering should reverse. It does, but for **one arm and under a conjunction of three conditions**. At our own setting (ϕ=0.6, 40 sessions) Γ_argmax still loses (d_z=-1.07). Stretch the horizon to 400 and it passes the model-based estimator ($+0.076$, d_z=$+2.09$). Add severe adversity and it beats *both* ($+0.044$ and $+0.031$, d_z=$+0.73$ and $+0.61$) while dropping out more often — so among survivors the margin is wider still. The reversal belongs to the deterministic ablation: Γ`_learned` scores 0.041 at that cell and never overtakes the model-free estimator at any horizon or adversity level (Appendix J). We therefore offer the sixth condition as *suggested* by these results rather than established: the reversal is post-hoc, confined to one arm, and requires a conjunction we did not preregister. Establishing it would take a preregistered factorial over bankability, horizon, adversity and policy type.

A caution: post-hoc test of a post-hoc diagnosis, outside the committed predictions. But it could have failed, and a first version did. Merely lengthening the horizon, with gain still accumulative, changed nothing. Duration was never the variable. Bankability was.

7.5 What the methodology caught

Several of our own claims fell here, each to one methodological choice. An unpaired test on a paired design hid a real signal effect. A single tuning search invented a non-monotonic ordering that five searches erased, and a three-seed run inflated an effect that five seeds halved. An unablated baseline let us credit homeostasis for what a rest rule was doing. Regulatory architectures make claims about *regimes* and are evaluated within-subject: dose–response sweeps, paired inference, multi-seed tuning with held-out validation, and ablation of the *baselines* — not only the proposal — should be default.

8 Limitations

The crossover is small, not the main agent's, and does not survive the stronger baseline (Sections 6.4, 6.5). At ϕ=1.2 all Γ arms lie between $+0.14$ and $+0.20$ against $+1.13$ for the estimator at ϕ=0: Γ wins by degrading less, not by performing well.

Signal validity is incompletely operationalised: only g_{prog}, against terminal expected gain, as a five-point correlation, with g_{conf} unvalidated. Relatedly $4a(1-a)$ peaks at a=0.5 while expected gain is maximised where accuracy is 0.354, capping fidelity by functional form. ϕ **is a compound**

knob — it moves fatigue severity, learning-rate degradation, error rates and dropout reachability together — so the sweep cannot attribute the differing degradation slopes to non-linearity specifically.

Policy tuning was a random search. 250 candidates show the original coefficients were poorly chosen; they do not find the best policy. The residual therefore also absorbs unsearched policy space, the fixed action representation, and any joint signal–policy optimum we missed. It is the gap unexplained by *the interventions we tested*, not a structural constant.

Researcher degrees of freedom remain: a self-attested hash, gate thresholds outside the committed text (though including the excluded point strengthens the contrast), and a post-hoc sixth condition. Condition (2) is untested. The simulator is simple, the domain single-task, and six visits per level is thin for estimating g — itself one of our findings.

9 Future Work

The decomposition sets the priority: with most of the gap unexplained by signal or coefficients, the next step is to **learn the policy**, not the signal. We are non-committal about the method — prior work in this line found value-based RL integrated poorly with homeostatic drives, for reasons we do not yet understand — so direct policy search over the coefficient vector is the conservative first step, with evolutionary methods natural given how few parameters the operator has. Since matched tuning shows signal and policy must be co-adapted, they should be searched jointly, and condition (4) needs a test granting Γ a genuinely private signal.

A third question follows from our own null: if a learned g matches but does not beat a constant, the culprit may be that it *keeps* learning. A constant has zero variance; a signal estimated from six visits per level injects noise into the drives at every step, indefinitely. Would a **convergence-stopped** signal — annealing $\eta \to 0$, or freezing g once its moving average stabilizes — keep the bias reduction while shedding the variance?

Most consequential is allostasis. Homeostasis corrects after deviation; allostasis anticipates it [2]. The jump demands a **world model**: a learned predictor of environmental dynamics and of how actions propagate through them, rolled forward before committing. Not a self-model — the regularities live in the world. Harder material produces errors at a rate set by the gap to ability. Effort compounds non-linearly. Recovery has a threshold. The agent authors none of these laws and must infer them; its drives are merely where they register. Knowing your own state predicts nothing unless you also know how the world will act on it. That is what the Good Regulator Theorem demands, and the system to model is the coupled pair, not the agent alone. World model as mechanism, feedforward as implementation, allostasis as principle — and our residual is precisely what it would have to explain.

10 Conclusion

Can a homeostatic agent learn its satiation signal from observable consequences and use it to regulate toward an external goal? Analyzed as the paired design it is: signal quality matters, and matters least. Under a fixed hand-chosen policy, a perfect signal is worth 7.7% of the gap to a simple estimator and six policy coefficients 25.0%. But searching the policy harder triples what the signal is worth and flips their interaction positive: the two are complementary, and a signal is worth little until the channel can carry it. Around 59% of the gap outlives both interventions — unexplained by what we tested, not proven structural.

Diagnosing why sent us back to our own framework, and we found it under-specified. Of five conditions, only one — that the agent hold what an outside observer cannot — makes homeostatic architecture necessary rather than merely applicable, and our domain fails it. We add a sixth: the objective must be maintenance, not a terminal payoff. Restore both and the ordering reverses.

Knowing what you need is not what limits you. The open problem is the channel, not the signal.

References

[1] **Cannon, W.B.** (1932). *The Wisdom of the Body*. New York: W.W. Norton.

[2] **Sterling, P. & Eyer, J.** (1988). Allostasis: A new paradigm to explain arousal pathology. In S. Fisher & J. Reason (eds.), *Handbook of Life Stress, Cognition and Health*, pp. 629–649. New York: Wiley.

[3] **Keramati, M. & Gutkin, B.S.** (2014). Homeostatic reinforcement learning for integrating reward collection and physiological stability. *eLife*, 3, e04811.

[4] **Ashby, W.R.** (1952). *Design for a Brain*. London: Chapman & Hall.

[5] **Dayan, P.** (1993). Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4), 613–624.

[6] **Gershman, S.J., Moore, C.D., Todd, M.T., Norman, K.A., & Sederberg, P.B.** (2012). The successor representation and temporal context. *Neural Computation*, 24(6), 1553–1581.

[7] **Lehnert, L. & Littman, M.L.** (2019). Successor features combine elements of model-free and model-based reinforcement learning. *Journal of Machine Learning Research*, 20(1), 1–53.

[8] **Zubek, R.** (2010). Needs-Based AI. In A. Lake (ed.), *Game Programming Gems 8*, pp. 231–238. Charles River Media.

[9] **Vygotsky, L.S.** (1978). *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press.

[10] **Rasch, G.** (1960). *Probabilistic Models for Some Intelligence and Attainment Tests*. Copenhagen: Danish Institute for Educational Research.

A Environment specification

Ability follows a Rasch model [10], $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning gain per exercise is

$$\Delta h = 0.08 \exp\left(-\frac{(\text{gap} - 0.5)^2}{0.5}\right) \cdot c \cdot \text{lr}_{\text{eff}}, \qquad \text{gap} = k - h_{\text{eff}},\ c \in \{1.0, 0.3\}$$

so failing at something challenging still teaches. Effort accumulates as $e \leftarrow 0.95e + \ell/20$ with ℓ the response latency, normalized by E_{ref}=3.0. Fatigue acts on two channels:

$$h_{\text{eff}} = \max(0.5,\ h - \phi e_n^{1.5}), \qquad \text{lr}_{\text{eff}} = \text{lr} \cdot \max(0.15,\ 1 - 0.7\phi e_n^{1.5})$$

The second is essential: without it, fatigue merely widens the difficulty gap and therefore moves the agent *toward* the bell curve's peak, so a fatigued student would learn *more* per session — an artefact that renders any fatigue sweep uninterpretable. Rest multiplies effort by 0.7, or 0.93 once e_n exceeds 1.3 (hysteresis), and costs 0.01 of ability. Frustration rises 0.25 per error, 0.3 more when failing at a mastered level, falls 0.35 on accessible successes, halves on rest, and triggers absorbing dropout at $0.95\times$ threshold for three consecutive sessions. Profiles draw $h_0 \in [1.5, 3.5]$, lr $\in [0.7, 1.3]$, threshold $\in [1.8, 3.0]$.

B Agent specification

Configuration: λ_p=0.15, λ_c=0.02, θ=0.7, α_p=α_c=0.3, w_{cp}= $-$ 0.05, w_{pc}=0.05, η=0.3, β=0.2.

```
def select(self, rng, env):
    dp = self.d_prog - theta                        # progress deviation
    dc = self.d_conf - theta                        # confidence deviation
    fn = self.frust / self.profile["umbral"]        # normalized frustration
    strain = max(0.0, fn - 0.7) / 0.3               # rest gate

    if rest_mode == "gated":                        # default arm
        l_rest = 2*max(0.0, dc)*strain + 4*strain
    elif rest_mode == "none":                       # viability ablation
        l_rest = -1e9
    else:                                           # drive-driven ablation
        l_rest = 3.0*max(0.0, dc) + 2.0*strain - 1.2

    logits = [c0*dp + c1*dc,                        # UP
              c2*(abs(dp) + abs(dc)) + c3,          # STAY
              c4*dc + c5*dp,                        # DOWN
              l_rest]                               # REST
    return ACTIONS[argmax(logits) if action_mode=="argmax"
                   else sample_softmax(logits, rng)]
```

In the default (gated) mode every term of `l_rest` is multiplied by `strain`, so the drives do not participate in the rest decision: along the viability dimension the agent is as reactive as the baselines it is compared against. This is why the viability claim of Section 7.2 is restricted to the `restdrive` arm.

C Quality gate across the sweep

Table 1: Gate criteria at each sweep point. **Key take-away:** the environment stops discriminating at ϕ=1.6, where the level-1 trap dissolves.

ϕ	always1	always5 drop	random	oracle	margin
0.0	+0.010	100%	+0.596	+1.139	+0.544 (PASS)
0.3	+0.035	100%	+0.307	+0.868	+0.561 (PASS)
0.6	+0.073	100%	+0.182	+0.533	+0.351 (PASS)
0.9	+0.108	100%	+0.133	+0.406	+0.273 (PASS)
1.2	+0.144	100%	+0.107	+0.357	+0.250 (PASS)
1.6	+0.162	100%	+0.088	+0.316	+0.228 (FAIL)

Only the first criterion fails at ϕ=1.6 ($+0.162$ against a 0.15 cutoff); the oracle margin remains healthy.

Sensitivity to the exclusion. At ϕ=1.2, Γ_argmax minus estimator is $+0.045$, CI $[+0.028, +0.062]$, d_z=$+0.52$; at the excluded ϕ=1.6 it is $+0.089$, CI $[+0.078, +0.101]$, d_z=$+1.54$. Including the point strengthens the claim, so the exclusion is conservative.

D Full main sweep (all ten arms)

Table 2: Learning gain / % dropout for every arm at every valid sweep point.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	+0.248/0%	+0.148/0%	+0.138/0%	+0.146/1%	+0.143/1%
Γ_argmax	+0.256/0%	+0.184/0%	+0.179/0%	+0.199/0%	+0.201/0%
Γ_restdrive	+0.363/4%	+0.186/7%	+0.165/12%	+0.161/15%	+0.154/16%
Γ_fixed	+0.271/0%	+0.142/0%	+0.148/0%	+0.151/0%	+0.147/0%
Γ_naive	+0.269/0%	+0.164/0%	+0.139/0%	+0.150/0%	+0.147/0%
Γ_oracle	+0.335/0%	+0.149/0%	+0.151/0%	+0.150/0%	+0.145/0%
rule	+0.591/0%	+0.413/0%	+0.270/0%	+0.220/0%	+0.199/0%
rule_norest	+0.595/0%	+0.417/1%	+0.278/13%	+0.229/38%	+0.207/55%
estimator	+1.132/0%	+0.700/0%	+0.360/0%	+0.228/0%	+0.156/0%
estimator_norest	+1.108/16%	+0.612/63%	+0.317/94%	+0.207/100%	+0.156/100%

E Additive decomposition

Table 3: Components of the ϕ=0 gap. **Left pair: the held-out estimate used in the abstract, body and conclusions** — coefficients selected on a 20-profile scoring subset and evaluated on 50 profiles never used for tuning. **Right pair: the earlier partly in-sample estimate**, retained only to show what validation leakage costs. **Key take-away:** leakage inflates the policy term by about three points and the ordering is unchanged.

Component	held-out	%	in-sample	%
Signal quality (oracle − constant, hand policy)	+0.065	7.7	+0.063	8.6
Policy coefficients (re-tuned − hand, constant g)	+0.211	25.0	+0.238	27.7
Interaction	-0.018	-2.1	-0.047	-5.5
Residual (estimator − best Γ)	+0.588	69.5	+0.606	70.5
Total gap vs. model-based estimator	+0.846	100.0	+0.860	100.0
held-out, vs. model-free estimator (total $+0.576$): 11.2, 36.6, -3.1, 55.2%				

F Signal-quality contrast across adversity

Table 4: P5: oracle versus constant g, paired over 100 profiles.

ϕ	oracle	constant	diff	95% CI	p	d_z
0.0	+0.335	+0.271	+0.063	[+0.055, +0.071]	6.4e-29	+1.59
0.3	+0.149	+0.142	+0.007	[+0.003, +0.011]	6.0e-04	+0.35
0.6	+0.151	+0.148	+0.003	[+0.000, +0.006]	2.0e-02	+0.24
0.9	+0.150	+0.151	-0.001	[-0.004, +0.001]	2.7e-01	-0.11
1.2	+0.145	+0.147	-0.002	[-0.004, -0.000]	3.2e-02	-0.22

Table 5: TOST equivalence p-values for oracle minus constant across candidate margins. **Key take-away:** from ϕ=0.3 onward the difference is inside every margin tested, so the conclusion does not depend on δ.

ϕ	diff	δ=0.02	δ=0.03	δ=0.05	δ=0.075	δ=0.10
0.0	+0.063	1.000	1.000	0.999	0.002	<.001
0.3	+0.007	<.001	<.001	<.001	<.001	<.001
0.6	+0.003	<.001	<.001	<.001	<.001	<.001
0.9	−0.001	<.001	<.001	<.001	<.001	<.001
1.2	−0.002	<.001	<.001	<.001	<.001	<.001

G Policy and crossover contrasts

Table 6: Paired contrasts over 100 profiles. Left: argmax minus softmax (P4). Right: Γ_argmax minus the model-based estimator (P3).

	P4: argmax − softmax			P3: Γ_argmax − estimator		
ϕ	diff	95% CI	d_z	diff	95% CI	d_z
0.0	+0.008	[+0.000, +0.016]	+0.21	-0.876	[-0.908, -0.843]	-5.29
0.3	+0.036	[+0.029, +0.043]	+1.00	-0.517	[-0.562, -0.472]	-2.27
0.6	+0.041	[+0.035, +0.048]	+1.25	-0.181	[-0.216, -0.146]	-1.02
0.9	+0.053	[+0.048, +0.058]	+1.98	-0.029	[-0.054, -0.004]	-0.23
1.2	+0.058	[+0.054, +0.061]	+3.23	+0.045	[+0.028, +0.062]	+0.52

H Objective-model ablation

Table 7: Learning gain with and without the objective model. **Key take-away:** removing privileged knowledge of the learning curve costs the estimator in the benign regime but *helps* it under adversity — and it still beats the homeostatic agent everywhere.

ϕ	estimator	estimator, no objective model	Γ_learned
0.0	+1.132	+0.851	+0.248
0.6	+0.360	+0.474	+0.138
1.2	+0.156	**+0.275**	+0.143

I Viability ablation

Table 8: Dropout % with and without a rest mechanism on *both* sides. **Key take-away:** without rest both architectures collapse, the regulated one slightly later; with rest both are safe. Viability tracks the presence of a rest mechanism, not of homeostatic drives.

	no rest mechanism		with rest mechanism	
ϕ	Γ_learned	estimator	Γ_learned	estimator
0.0	6.4%	16.4%	0.0%	0.0%
0.6	81.5%	94.5%	0.4%	0.0%
1.2	93.5%	100.0%	1.0%	0.0%

J Maintenance-objective experiment

The primary task has a terminal objective: accumulate ability by session 40. Gain can be banked, so more is always better. To test whether homeostatic regulation fares better under a *maintenance*

objective we replace it with **mean readiness** — $\sigma(1.5(h_{\text{eff}} - 3))$, the score the student would obtain if the exam were held that session, averaged over the horizon, with zero credit for every session after quitting. Readiness cannot be banked: fatigue costs continuously rather than only at a deadline.

Table 9: Mean readiness over 60 profiles $\times$ 6 seeds. **Key take-away:** the ordering reverses only under the conjunction of a non-bankable objective, a long horizon, and severe adversity — at the paper's own setting (top row) the estimators still win.

ϕ	horizon	Γ_learned	Γ_argmax	estimator	estimator (no obj.)	rule
0.6	40	0.158	0.141	0.203	**0.231**	0.178
0.6	100	0.110	0.103	0.122	**0.168**	0.115
0.6	200	0.119	0.134	0.102	**0.160**	0.097
0.6	400	0.145	0.196	0.120	**0.210**	0.122
1.2	40	0.099	0.084	0.102	**0.114**	0.098
1.2	100	0.054	0.049	0.055	**0.061**	0.053
1.2	200	0.038	0.040	0.039	**0.043**	0.039
1.2	400	0.041	**0.080**	0.035	0.049	0.037

Paired contrasts for Γ_argmax, n=60 profiles: at ϕ=0.6, H=40 it trails the estimator by -0.062 (d_z=-1.07) and the model-free estimator by -0.090 (d_z=-1.29). At ϕ=0.6, H=400 it leads the estimator by $+0.076$ (d_z=$+2.09$) but trails the model-free one by -0.014. At ϕ=1.2, H=400 it leads both, by $+0.044$ (d_z=$+0.73$) and $+0.031$ (d_z=$+0.61$), all $p < 10^{-4}$.

A failed first attempt. We first tested the hypothesis by extending the horizon alone while keeping cumulative gain as the objective. That test failed: no baseline ever quit, the estimator's advantage *grew* with horizon, and Γ's survival fell to 85% at 400 sessions. Lengthening a horizon does not make an accumulative objective a maintenance objective, which is the distinction the sixth condition turns on. We report the failed version because it is what identified the right variable.

K Matched-tuning results

Table 10: Held-out validation gain by signal and search budget; five independent search seeds at each budget, mean (sd). Last column: oracle − constant, and learned − constant. **Key take-away:** how much signal quality is worth is not a property of the signal — it roughly doubles when the policy is searched harder.

Signal	250 candidates, 5 seeds	1200 candidates, 3 seeds	oracle − constant
constant g	$+0.472$ (0.051)	$+0.487$ (0.060)	—
learned g	$+0.480$ (0.068)	$+0.581$ (0.059)	$+0.008 \rightarrow +0.094$
oracle g	$+0.519$ (0.052)	$+0.608$ (0.036)	$+0.047 \rightarrow +0.121$

Per-search held-out validation gains, 50 profiles never used for tuning. At 250 candidates: constant $\{.464, .458, .473, .412, .554\}$, learned $\{.439, .455, .599, .471, .438\}$, oracle $\{.509, .569, .499, .447, .570\}$ — oracle above constant in $5/5$. At 1200 candidates: constant $\{.411, .466, .478\}$, learned $\{.545, .680, .527\}$, oracle $\{.656, .569, .604\}$ — oracle and learned each above constant in $3/3$. Paired contrasts at the 250 budget (n=50 profiles, averaging over search seeds): learned − constant = $+0.008$, CI $[-0.002, +0.018]$, p=.11, TOST $p < 10^{-4}$ (equivalent); oracle − constant = $+0.047$, CI $[+0.036, +0.058]$, d_z=$+1.22$; oracle − learned = $+0.039$, d_z=$+1.15$.

Table 11: Held-out validation gain across five independent coefficient searches per signal. **Key take-away:** the ordering *is* monotone in signal quality once search variance is accounted for — the earlier non-monotonic result was a single-search artefact — but the spread is small.

Signal	mean (sd) over 5 searches	larger budget (1200)	vs. constant
constant g	$+0.472$ (0.051)	$+0.411$	—
learned g	$+0.480$ (0.068)	$+0.545$	indistinguishable
oracle g	$+\mathbf{0.519}$ (0.052)	$+\mathbf{0.656}$	higher in **5/5** searches
estimator: $+1.132$*; estimator without objective model:* $+0.851$			

L Policy coefficient search

The four action logits are linear in d_p, d_c with six free coefficients. The original values $(4, -3, -3, 1.5, 4, -3)$ were chosen by hand. To test whether performance is limited by these particular numbers we ran a random search over 250 candidates sampled from $c_0, c_4 \sim U(0, 10)$; $c_1, c_5 \sim U(-10, 2)$; $c_2 \sim U(-10, 0)$; $c_3 \sim U(-2, 5)$, scored on a 30-profile subset and validated on 100 profiles. The matched protocol of Section 6.3 repeats this independently per signal with five search seeds and a strictly disjoint 50-profile validation set; those held-out numbers, not the ones below, support the paper's conclusions.

Table 12: Effect of coefficient re-tuning at ϕ=0, shared search, validated on 100 profiles including the 30 used for scoring (partly in-sample; see Section 6.3 for the held-out version).

Arm	hand-chosen	re-tuned	Δ
Γ_oracle	+0.335	+0.526	+0.191
Γ_fixed	+0.271	+0.509	+0.238
Γ_learned	+0.248	+0.266	+0.018
estimator: $+1.132$			

M Shape-shift results and quality gate

Table 13: Peak of the learning curve moved at session 20; ϕ=0.6; 100 paired profiles. **Key take-away:** all four points pass the gate, and the estimator's lead grows as its own model is corrupted.

peak	Γ_argmax	Γ_oracle	estimator	gap	d_z	gate margin
0.5	+0.179	+0.151	+0.360	-0.181	-1.02	+0.370 (PASS)
1.0	+0.145	+0.110	+0.345	-0.200	-1.19	+0.316 (PASS)
1.5	+0.114	+0.068	+0.324	-0.210	-1.32	+0.294 (PASS)
2.0	+0.103	+0.057	+0.301	-0.198	-1.26	+0.280 (PASS)

N Complete sweep metrics

Table 14: Time-in-band, *conditional on study sessions* rather than on the 40-session budget. **Caveat:** the regulated arms rest far more often than the baselines, so this denominator omits the opportunity cost of rest and flatters high-rest policies; the full-horizon version is computed in the released code as `tib_full`.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	0.271	0.350	0.519	0.610	0.644
Γ_argmax	0.252	0.340	0.541	0.670	0.715
Γ_restdrive	0.289	0.343	0.519	0.578	0.597
Γ_fixed	0.229	0.354	0.571	0.637	0.656
Γ_naive	0.250	0.341	0.498	0.610	0.648
Γ_oracle	0.279	0.400	0.599	0.647	0.660
rule	0.364	0.409	0.498	0.612	0.670
rule_norest	0.365	0.414	0.510	0.613	0.662
estimator	0.805	0.780	0.592	0.459	0.381
estimator_norest	0.796	0.749	0.589	0.447	0.371

Table 15: Secondary diagnostic: terminal exam score on effective ability, *not* dropout-adjusted — arms that quit still receive a score, so cells with high dropout are not comparable. **Key take-away:** the ordering does not always follow learning gain, which is why we fix gain as the primary outcome rather than treating the exam as the objective.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	0.446	0.205	0.085	0.041	0.031
Γ_argmax	0.448	0.192	0.072	0.038	0.028
Γ_restdrive	0.484	0.169	0.055	0.028	0.024
Γ_fixed	0.454	0.164	0.047	0.028	0.023
Γ_naive	0.454	0.189	0.071	0.033	0.027
Γ_oracle	0.474	0.150	0.039	0.025	0.023
rule	0.556	0.251	0.087	0.039	0.026
rule_norest	0.557	0.250	0.083	0.035	0.024
estimator	0.716	0.338	0.084	0.034	0.024
estimator_norest	0.708	0.290	0.073	0.031	0.025

Table 16: Mean REST sessions out of 40. **Key take-away:** the regulated arms spend much of the budget resting — the price of the viability they buy.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	9.1	8.0	6.7	6.2	6.0
Γ_argmax	8.8	6.5	4.5	3.4	3.0
Γ_restdrive	5.0	4.7	4.4	4.2	4.2
Γ_fixed	6.7	5.7	5.0	5.0	5.0
Γ_naive	7.3	6.7	5.9	5.6	5.6
Γ_oracle	6.3	5.7	5.1	5.1	5.1
rule	0.1	0.3	0.8	1.5	2.0
rule_norest	0.0	0.0	0.0	0.0	0.0
estimator	1.1	2.4	4.0	5.0	5.4
estimator_norest	0.0	0.0	0.0	0.0	0.0

Table 17: Satiation-signal validity. **Key take-away:** validity degrades with adversity for the learned arm but not the oracle.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	+0.493	+0.415	+0.351	+0.303	+0.281
Γ_argmax	+0.550	+0.443	+0.505	+0.456	+0.452
Γ_restdrive	+0.441	+0.488	+0.470	+0.431	+0.397
Γ_fixed	—	—	—	—	—
Γ_naive	—	—	—	—	—
Γ_oracle	+0.994	+0.861	+0.896	+0.920	+0.909
rule	—	—	—	—	—
rule_norest	—	—	—	—	—
estimator	—	—	—	—	—
estimator_norest	—	—	—	—	—

O Full sweep figure

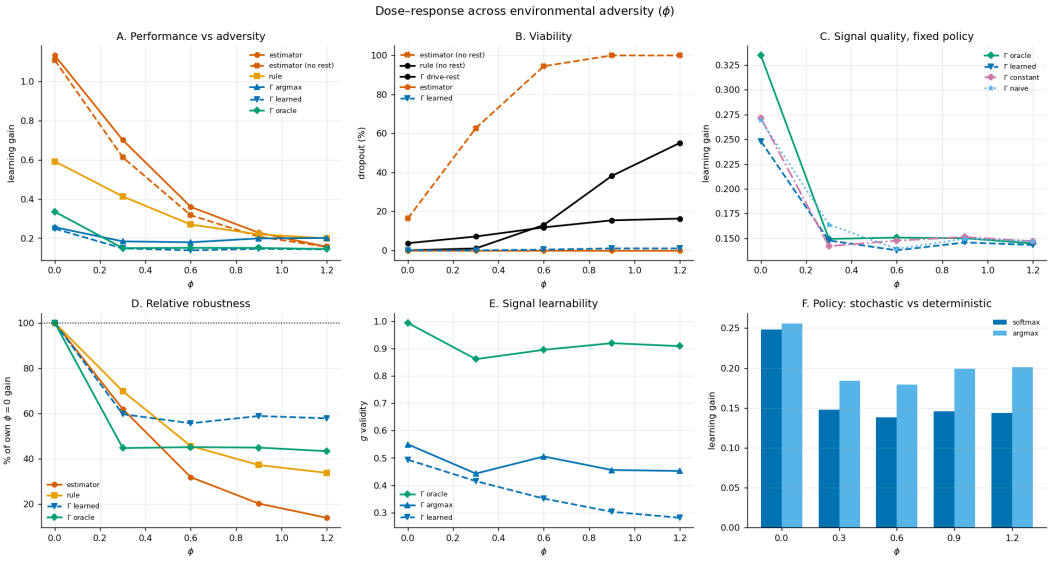

Figure 1: Six-panel summary, Okabe-Ito colourblind-safe palette with distinct markers and line styles so the panels remain legible in greyscale. **A**: performance across adversity. **B**: viability. **C**: signal quality under the fixed hand policy. **D**: robustness normalized to each arm's own ϕ=0 value. **E**: signal learnability. **F**: stochastic versus deterministic policy.

P Compute and LLM usage

All experiments run on a commodity laptop with no GPU; the main sweep of 60,000 episodes takes 31.4 s single-threaded and a single decision costs 14 μs (full specifications in Appendix Q).

We disclose LLM usage for transparency, although it is not a component of the core method: no language model participates in the agent, the environment, the baselines, or the analysis, all of which are explicit numerical simulations. LLMs served as engineering assistants in two capacities — code assistance, via Devin as an IDE together with Kimi 3 and DeepSeek v4 Pro, and Claude Opus 5 and Claude Fable 5; and writing and editing assistance, via Claude Opus 5 and Claude Fable 5. All experimental design decisions, the committed predictions, the statistical procedures, and every interpretation are the authors' own, and all reported numbers were produced by executing the released notebook.

Q Compute resources

Intel Core i7-1165G7, 4 cores, 32 GB RAM, **no GPU**, pure Python with `scipy` and `matplotlib` as the only non-standard dependencies. The main sweep (60,000 episodes across 6 adversity points and 10 arms) takes 31.4 s single-threaded; the multi-seed coefficient search adds roughly 5 minutes; the maintenance-objective experiment about 4 minutes. Per-episode cost is 0.55 ms for 40 decisions, i.e. 14 μs per decision, consistent with the operator's $O(n^2 + A)$ per-session arithmetic and absence of gradients, value functions, or replay. The entire project consumed less compute than a single GPU-minute.

R Reproducibility

Ten seeds are derived from master seed `20260820` via SHA-256, each mapped to the next prime above $10007 + (\text{digest} \bmod 900000)$; profiles come from a separate fixed seed. The anonymized repository at `https://anonymous.4open.science/r/gamma-n2-policy-bottleneck-0000` reproduces every reported number: `python run_all.py` runs the full pipeline in about ten minutes on a laptop with no GPU (`-full` adds the 1200-candidate tuning sweep, about thirty-five). Each experiment is a standalone script that prints the paper numbers it produces and writes JSON to `results/`; `README.md` maps every claim in the paper to the script that generates it. The first step verifies the prediction commitment hash and aborts if `predictions.txt` has been edited. Dependencies are `scipy` and `matplotlib`.

S Preregistered prediction text (verbatim)

The authoritative artifact is `predictions.txt` in the supplement, whose SHA-256 digest is `6e888c7e46e1dd306b6adc601a45cc08ff914a24d2f474a913a1ff2907bfc084`. It contains one prediction per line in sorted key order, joined with newlines. Below the same text is shown with display-only wrapping; continuation lines are indented and are *not* part of the hashed string.

```
P1: bayesian_norest (estimacion sin gestion de viabilidad) colapsa al subir
    phi: dropout alto, gain al piso.
P2: Los brazos Gamma se degradan proporcionalmente menos que el bayesiano a
    lo largo del barrido de phi.
P3: Existe phi* dentro del rango valido de la puerta de calidad donde algun
    brazo Gamma supera al bayesiano en gain.
P4: gamma_learned_argmax supera a gamma_learned en todos los puntos validos
    del barrido.
P5: Si la calidad de g se transmite a la conducta, gamma_oracle se separa de
    gamma_no_learning en algun punto del barrido.
P6: g_validity de gamma_learned decrece monotonamente al subir phi.
P7: Bajo desplazamiento del PICO de la campana de aprendizaje (cambio de
    forma, no de escala), los brazos Gamma ganan terreno relativo frente al
    bayesiano, que queda mal especificado y no puede absorberlo re-estimando h.
```

T Source code

Self-contained: no external dataset, no pretrained model, only `scipy` and `matplotlib` beyond the standard library. **Naming note:** the code predates the paper's terminology — `BayesianAgent` is the paper's *estimator*, `BayesianNoObjAgent` the estimator without an objective model, `ReglaAgent` the *rule*, `gamma_no_learning` the *constant-g* arm; `nivel`, `ganancia` and `descansar` are level, gain and rest.

T.1 Seeds and determinism

```
import hashlib, math, random
from collections import defaultdict

NIVELES=[1,2,3,4,5]
PRESUPUESTO=40
E_REF=3.0

```

```
def _is_prime(n):
    if n<2: return False
    if n in (2,3,5,7): return True
    if n%2==0 or n%3==0: return False
    i=5
    while i*i<=n:
        if n%i==0 or n%(i+2)==0: return False
        i+=6
    return True
def _next_prime(n):
    if n<=2: return 2
    c=n|1
    while not _is_prime(c): c+=2
    return c
def derive_seeds(master,n=10):
    s=[]
    for i in range(n):
        d=hashlib.sha256(f"{master}:run_{i}".encode()).hexdigest()
        s.append(_next_prime(10007+int(d,16)%900000))
    return s

def rasch_p(h,nivel): return 1.0/(1.0+math.exp(-1.5*(h-nivel)))
PEAK_MODEL=0.5   # peak assumed by any agent that models the curve (misspecifiable)
def bell_curve(gap,peak=PEAK_MODEL): return 0.08*math.exp(-((gap-peak)**2)/0.5)
```

T.2 Environment

```
class StudyEnv:
    """v4: fatiga normalizada + histeresis + shift de regimen + examen en h_eff."""
    def __init__(self, h0, lr_mult=1.0, phi=0.0, lambda_olv=0.01,
                 hysteresis=False, e_crit=1.3, shift_mult=1.0, shift_at=20,
                 peak_gap=PEAK_MODEL, peak_new=None, peak_at=20):
        self.h=h0; self.h0=h0; self.lr_mult=lr_mult
        self.phi=phi; self.phi0=phi
        self.lambda_olv=lambda_olv
        self.hysteresis=hysteresis; self.e_crit=e_crit
        self.shift_mult=shift_mult; self.shift_at=shift_at
        self.peak_gap=peak_gap; self.peak_new=peak_new; self.peak_at=peak_at
        self.esfuerzo=0.0

    def tick(self,s):
        if self.shift_mult!=1.0 and s==self.shift_at:
            self.phi=self.phi0*self.shift_mult
        if self.peak_new is not None and s==self.peak_at:
            self.peak_gap=self.peak_new

    @property
    def esf_n(self): return self.esfuerzo/E_REF
    @property
    def h_eff(self): return max(0.5, self.h - self.phi*self.esf_n**1.5)
    @property
    def lr_eff(self):
        # la fatiga degrada la CAPACIDAD de aprender, no solo la dificultad aparente
        return self.lr_mult*max(0.15, 1.0 - 0.7*self.phi*self.esf_n**1.5)

    def expected_gain(self,nivel):
        gap=nivel-self.h_eff; p=rasch_p(self.h_eff,nivel)
        return bell_curve(gap,self.peak_gap)*(p*1.0+(1-p)*0.3)*self.lr_eff
    def in_zpd(self,nivel):
        g=[self.expected_gain(k) for k in NIVELES]; mx=max(g)
        return (self.expected_gain(nivel)>=0.5*mx) if mx>0 else False

    def exercise(self,nivel,rng):
        p=rasch_p(self.h_eff,nivel)
        correcto=rng.random()<p
        gap=max(0.0,nivel-self.h_eff)
        latencia=rng.lognormvariate(math.log(5)+0.6*gap,0.4)
        self.esfuerzo=0.95*self.esfuerzo+latencia/20.0
```

```
        gain=bell_curve(nivel-self.h_eff,self.peak_gap)*(1.0 if correcto else 0.3)*self.lr_eff
        self.h=min(6.0,self.h+gain)
        return {"correcto":correcto,"latencia":round(latencia,2),"gain":gain}

    def descansar(self):
        if self.hysteresis and self.esf_n>self.e_crit: self.esfuerzo*=0.93
        else: self.esfuerzo*=0.7
        self.h=max(0.5,self.h-self.lambda_olv)
```

T.3 Homeostatic agent

The coupling uses the *receiving* drive's pressure, matching Eq. (1); the legacy sender-modulated branch is kept for reference only and used for no reported result.

```
class GCfg:
    def __init__(self,**kw):
        self.lam_p=kw.get("lam_p",0.15); self.lam_c=kw.get("lam_c",0.02)
        self.theta=kw.get("theta",0.7)
        self.alpha_p=kw.get("alpha_p",0.3); self.alpha_c=kw.get("alpha_c",0.3)
        self.w_cp=kw.get("w_cp",-0.05); self.w_pc=kw.get("w_pc",0.05)
        self.g_mode=kw.get("g_mode","learned")
        self.n_drives=kw.get("n_drives",2); self.coupling=kw.get("coupling",True)
        self.coupling_mode=kw.get("coupling_mode",
                "receiver")  # receiver (Eq.1 / design intent) | sender (legacy bug)
        self.eta=kw.get("eta",0.3); self.beta=kw.get("beta",0.2)
        self.g_init_p=kw.get("g_init_p",0.3); self.g_init_c=kw.get("g_init_c",0.5)
        self.g_naive=kw.get("g_naive",0.5)
        self.action_mode=kw.get("action_mode","softmax")   # softmax | argmax
        self.rest_mode=kw.get("rest_mode","gated")  # gated | drive | none
        self.coef=kw.get("coef",(4.0,-3.0,-3.0,1.5,4.0,-3.0))  # up_p,up_c,stay_k,stay_b,down_c,
                down_p          # gated | drive

class GammaAgent:
    def __init__(self,cfg,profile):
        self.cfg=cfg; self.p=profile
        self.d_prog=0.5; self.d_conf=0.5
        self.g_prog={k:cfg.g_init_p for k in NIVELES}
        self.g_conf={k:cfg.g_init_c for k in NIVELES}
        self.acc_ema={}; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
    def _pressure(self,d): return min(1.0,abs(d-self.cfg.theta)/0.8)
    def basal(self):
        m=self.acc_ema.get(self.nivel,0.5)
        self.d_prog=min(2.0,self.d_prog+self.cfg.lam_p*m)
        if self.cfg.n_drives>=2:
            self.d_conf=min(2.0,self.d_conf+self.cfg.lam_c)
            if self.cfg.coupling:
                pc=self._pressure(self.d_conf); pp=self._pressure(self.d_prog)
                if self.cfg.coupling_mode=="receiver":
                    # Eq.(1): modulated by the RECEIVING drive's own pressure
                    m_to_prog, m_to_conf = pp, pc
                else:
                    m_to_prog, m_to_conf = pc, pp     # legacy: sender's pressure
                if self.d_conf>self.cfg.theta+0.05:
                    self.d_prog=max(0.0,self.d_prog+self.cfg.w_cp*m_to_prog)
                if self.d_prog>self.cfg.theta+0.05:
                    self.d_conf=max(0.0,self.d_conf+self.cfg.w_pc*m_to_conf)
    def select(self,rng,env):
        th=self.cfg.theta
        dp=self.d_prog-th
        dc=(self.d_conf-th) if self.cfg.n_drives>=2 else 0.0
        fn=self.frust/max(self.p["umbral"],0.1)
        strain=max(0.0,fn-0.7)/0.3
        if self.cfg.n_drives>=2:
            if self.cfg.rest_mode=="gated":
                l_rest=2*max(0.0,dc)*strain+4*strain
            elif self.cfg.rest_mode=="none":
                l_rest=-1e9          # rest unavailable: isolates drives from the frustration
```

```
                        gate
            else:
                l_rest=3.0*max(0.0,dc)+2.0*strain-1.2
            c=self.cfg.coef
            logits=[c[0]*dp+c[1]*dc, c[2]*(abs(dp)+abs(dc))+c[3], c[4]*dc+c[5]*dp, l_rest]
        else:
            logits=[4*dp,-3*abs(dp)+1.5,-4*dp,4*strain]
        acts=["subir","mantener","bajar","descansar"]
        if self.cfg.action_mode=="argmax":
            return acts[max(range(4),key=lambda i:logits[i])]
        mx=max(logits); exps=[math.exp(min(50,l-mx)) for l in logits]
        tot=sum(exps); r=rng.random()*tot; cum=0.0
        for i,e in enumerate(exps):
            cum+=e
            if r<cum: return acts[i]
        return "mantener"
    def study(self,env,outcome,nivel):
        c=outcome["correcto"]; eb=self.acc_ema.get(nivel,0.5)
        self.acc_ema[nivel]=(1-self.cfg.beta)*eb+self.cfg.beta*float(c)
        if self.cfg.g_mode=="learned":
            a=self.acc_ema[nivel]
            self.g_prog[nivel]=(1-self.cfg.eta)*self.g_prog[nivel]+self.cfg.eta*(4.0*a*(1.0-a))
            self.g_conf[nivel]=(1-self.cfg.eta)*self.g_conf[nivel]+self.cfg.eta*a
        elif self.cfg.g_mode=="oracle":
            self.g_prog[nivel]=min(1.0,env.expected_gain(nivel)/0.042)
            self.g_conf[nivel]=rasch_p(env.h_eff,nivel)
        if self.cfg.g_mode=="naive": gp=gc=self.cfg.g_naive
        else: gp=self.g_prog[nivel]; gc=self.g_conf[nivel]
        if c:
            self.d_prog=max(0.0,self.d_prog-self.cfg.alpha_p*gp)
            if self.cfg.n_drives>=2:
                self.d_conf=max(0.0,self.d_conf-self.cfg.alpha_c*gc)
            if outcome["latencia"]<15: self.frust=max(0.0,self.frust-0.35)
        else:
            if self.cfg.n_drives>=2: self.d_conf=min(2.0,self.d_conf+0.1)
            self.frust+=0.25
            if eb>=0.7: self.frust+=0.3
```

T.4 Baselines

```
class ReglaAgent:
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
        self.streak_c=0; self.streak_e=0; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        if self.streak_e>=2: return "bajar"
        if self.streak_c>=3: return "subir"
        return "mantener"
    def study(self,env,outcome,nivel):
        if outcome["correcto"]: self.streak_c+=1; self.streak_e=0
        else: self.streak_e+=1; self.streak_c=0

class BayesianNoObjAgent:
    """State estimation WITHOUT an objective model: tracks h_hat from outcomes and
    simply matches the level to estimated ability. Isolates estimation from
    privileged knowledge of where the learning curve peaks."""
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
        self.frust=0.0; self.consec_high=0; self.abandono=False
        self.dropout_session=None; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        best=min(NIVELES,key=lambda k: abs(k-self.h_hat))
        if best>self.nivel: return "subir"
        if best<self.nivel: return "bajar"
        return "mantener"
```

```
    def study(self,env,outcome,nivel):
        p=rasch_p(self.h_hat,nivel)
        self.h_hat+=0.3*(float(outcome["correcto"])-p)
        self.h_hat=max(0.5,min(6.0,self.h_hat))

class BayesianAgent:
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
        self.frust=0.0; self.consec_high=0; self.abandono=False
        self.dropout_session=None; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        best,bg=self.nivel,-1
        for k in NIVELES:
            gap=k-self.h_hat; p=rasch_p(self.h_hat,k)
            g=bell_curve(gap)*(p+(1-p)*0.3)
            if g>bg: best,bg=k,g
        if best>self.nivel: return "subir"
        if best<self.nivel: return "bajar"
        return "mantener"
    def study(self,env,outcome,nivel):
        p=rasch_p(self.h_hat,nivel)
        self.h_hat+=0.3*(float(outcome["correcto"])-p)
        self.h_hat=max(0.5,min(6.0,self.h_hat))

class FixedAgent:
    def __init__(self,profile,mode):
        self.p=profile; self.mode=mode; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
        self.acc_ema={}
    def select(self,rng,env):
        if self.mode=="always1": t=1
        elif self.mode=="always5": t=5
        elif self.mode=="random": t=rng.choice(NIVELES)
        else: t=max(NIVELES,key=lambda k:env.expected_gain(k))
        if t>self.nivel: return "subir"
        if t<self.nivel: return "bajar"
        return "mantener"
    def study(self,env,outcome,nivel): pass

def shared_frust_update(agent,outcome,eb):
    if outcome["correcto"]:
        if outcome["latencia"]<15: agent.frust=max(0.0,agent.frust-0.35)
    else:
        agent.frust+=0.25
        if eb>=0.7: agent.frust+=0.3
```

T.5 Profiles and episode runner

```
def gen_profiles(n=100,seed=2026):
    rng=random.Random(seed)
    return [{"h0":round(rng.uniform(1.5,3.5),2),
             "lr_mult":round(rng.uniform(0.7,1.3),2),
             "umbral":round(rng.uniform(1.8,3.0),1)} for _ in range(n)]

def run_episode(agent,profile,seed,is_gamma,env_kw,log=False):
    env=StudyEnv(profile["h0"],lr_mult=profile["lr_mult"],**env_kw)
    rng=random.Random(seed); traj=[]; zpd=0; nst=0; ndesc=0; s=0
    for s in range(PRESUPUESTO):
        if agent.abandono: break
        env.tick(s)
        if is_gamma: agent.basal()
        a=agent.select(rng,env)
        if a=="subir": agent.nivel=min(5,agent.nivel+1)
        elif a=="bajar": agent.nivel=max(1,agent.nivel-1)
        out=None; inz=False
        if a=="descansar":
            env.descansar(); agent.frust*=0.5; ndesc+=1
```

```
        else:
            inz=env.in_zpd(agent.nivel)
            out=env.exercise(agent.nivel,rng); nst+=1
            if inz: zpd+=1
            if is_gamma: agent.study(env,out,agent.nivel)
            else:
                eb=agent.acc_ema.get(agent.nivel,0.5)
                agent.acc_ema[agent.nivel]=0.8*eb+0.2*float(out["correcto"])
                shared_frust_update(agent,out,eb)
                agent.study(env,out,agent.nivel)
        if agent.frust>=0.95*profile["umbral"]: agent.consec_high+=1
        else: agent.consec_high=0
        if agent.consec_high>=3 and not agent.abandono:
            agent.abandono=True; agent.dropout_session=s
        if log:
            traj.append({"s":s,"nivel":agent.nivel,"action":a,
                "correcto":out["correcto"] if out else None,
                "d_prog":round(getattr(agent,"d_prog",0),3),
                "d_conf":round(getattr(agent,"d_conf",0),3),
                "frust":round(agent.frust,2),"h":round(env.h,3),
                "h_eff":round(env.h_eff,3),"esf_n":round(env.esf_n,3),"in_zpd":inz})
    m={"ganancia":round(env.h-profile["h0"],3),
       "h_final":round(env.h,3),
       "exam":round(rasch_p(env.h,3),3),
       "exam_taken":round(0.0 if agent.abandono else rasch_p(env.h_eff,3),3),
       "exam_eff":round(rasch_p(env.h_eff,3),3),
       "esf_n_final":round(env.esf_n,3),
       "tiz":round(zpd/max(1,nst),3),
       "tib_full":round(zpd/PRESUPUESTO,3),
       "dropout":agent.abandono,"dropout_session":agent.dropout_session,
       "n_descansar":ndesc,"n_study":nst,"nivel_final":agent.nivel,
       "sessions_used":s+1}
    if is_gamma and agent.cfg.g_mode in ("learned","oracle","fixed","naive"):
        gains=[env.expected_gain(k) for k in NIVELES]
        gps=[agent.g_prog[k] for k in NIVELES]
        mg=sum(gains)/5; mp=sum(gps)/5
        cov=sum((gains[i]-mg)*(gps[i]-mp) for i in range(5))
        vg=math.sqrt(sum((g-mg)**2 for g in gains)); vp=math.sqrt(sum((g-mp)**2 for g in gps))
        m["g_validity"]=round(cov/(vg*vp),3) if vg>0 and vp>0 else None
        m["g_prog_final"]={str(k):round(agent.g_prog[k],3) for k in NIVELES}
        m["true_gain_final"]={str(k):round(env.expected_gain(k),4) for k in NIVELES}
    return m,traj

CFG_BASE=dict(lam_p=0.15,alpha_p=0.3,alpha_c=0.3,theta=0.7)
```

U Analysis code: paired inference and equivalence

The design is within-subject — the same 100 profiles face every arm — so contrasts are paired and equivalence is tested explicitly rather than inferred from non-significance.

```
from scipy.stats import ttest_rel, t as tdist

def per_profile(arm, phi):                  # one unit per profile
    return [mean(run(arm, profile, seed, phi) for seed in SEEDS)
            for profile in PROFILES]

def paired_contrast(a, b, phi):
    xa, xb = per_profile(a, phi), per_profile(b, phi)
    d  = [u - v for u, v in zip(xa, xb)]
    n  = len(d); m = mean(d); se = stdev(d) / sqrt(n)
    h  = se * tdist.ppf(0.975, n - 1)       # 95% CI half-width
    t, p = ttest_rel(xa, xb)                # NOT ttest_ind: paired design
    return m, (m - h, m + h), p, m / stdev(d)
```

```
def tost(d, delta):                          # two one-sided tests
    n = len(d); m = mean(d); se = stdev(d) / sqrt(n)
    return max(1 - tdist.cdf((m + delta) / se, n - 1),
                   tdist.cdf((m - delta) / se, n - 1))

# P3 is an existence claim: max-statistic sign-flip permutation
T_obs = max(paired_t(arm, phi) for arm in GAMMA_ARMS for phi in PHIS_VALID)
null  = [max(paired_t(arm, phi, flip=s) for arm in GAMMA_ARMS
             for phi in PHIS_VALID) for s in random_sign_flips(2000)]
p_P3  = (sum(t >= T_obs for t in null) + 1) / 2001
```

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction state the split answer (g is learnable; the learned signal does not reach behavior), the viability result, and the RLP decomposition, matching the Results section exactly. The abstract reports the three-way decomposition with effect sizes, states that a learned signal underperforms a constant, reports the refuted prediction P7, and states that the crossover result concerns robustness, and the Limitations section bounds the claim by noting the crossover occurs in a low-gain regime.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The dedicated Limitations section covers the low-gain regime in which the crossover occurs, incomplete operationalization of signal validity (only g_{prog}, terminal, five points), the functional-form mismatch that caps achievable validity and partially confounds the oracle contrast, the fact that policy tuning was a random search rather than an optimum (making the residual an upper bound), the still-untested applicability conditions (2) and (4), the self-attested commitment hash and the un-committed gate thresholds, and simulator simplicity.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper presents no theorems or formal proofs. Its contributions are empirical, and the equations given in the Architecture section and Appendix B are model definitions rather than theoretical results requiring proof.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendix A gives the complete environment specification with all constants; Appendix B gives the agent configuration and the action-selection code; Appendix F documents the seed derivation procedure (SHA-256 from master seed 20260820), the profile generation, and the compute requirements. The Experimental Design section specifies the arm definitions and the paired statistical protocol. Sample sizes differ by experiment and are stated separately: the main sweep uses 100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 adversity points (60,000 episodes); coefficient tuning scores on a 20-profile subset and validates on 50 strictly held-out profiles; the post-hoc maintenance experiment uses 60 profiles $\times$ 6 seeds. An anonymized reproducible notebook is included in the supplementary material, and the SHA-256 commitment hash of the preregistered predictions is reported in the paper and recomputable from the verbatim prediction text in the appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case

of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: An anonymized self-contained notebook reproducing every reported number, table, and figure is included in the supplementary material. No external dataset is required: the environment is a simulator fully specified in Appendix A, and all stochasticity is controlled by the documented seed derivation. Dependencies are limited to `scipy` and `matplotlib`.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The Experimental Design section specifies the adversity axis and its sweep values, the ten arms, the episode protocol (100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points),

and the profile-clustered statistical procedure. Appendix B lists every agent hyperparameter. The same section documents the quality gate applied at each sweep point and states that results are reported only over the passing range.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The design is within-subject (the same 100 profiles face every arm), so all inferential comparisons use paired t-tests on profile-level means (n=100), reporting the paired difference, its 95% confidence interval, and the paired effect size d_z. Claims resting on the absence of an effect are additionally supported by TOST equivalence tests against a prespecified smallest effect size of interest (δ=0.05), rather than inferred from non-significance. Confidence intervals are reported for every headline contrast, including the crossover. The clustering unit and test are stated in the Experimental Design section.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: A dedicated Compute Resources section states the machine (Intel Core i7-1165G7, 4 cores, 32 GB RAM, no GPU), the wall-clock time for the main sweep (60,000 episodes in 31.4 s single-threaded), the coefficient search (3 min), and the per-decision cost (14 μs), together with the operator's $O(n^2 + A)$ per-session arithmetic and its lack of gradients, value functions, or replay. Exploratory and discarded runs during development were of the same negligible per-run cost, and the entire project consumed less compute than a single GPU-minute.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The research uses only a synthetic simulator with no human subjects, no personal data, and no scraped assets. Anonymity is preserved throughout the submission. Potential dual-use concerns arising from modeling learner internal states are discussed in the Impact Statement.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Impact Statement section discusses the positive case (tutoring systems that respect cognitive and emotional limits, supported directly by the paper's viability results) and the negative case (the same mechanism inverted into an engagement-maximizing system that learns when to intervene to keep a user hooked), and identifies transparency about whose homeostasis is regulated as the mitigation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.

- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released artifact is a small simulation notebook implementing a synthetic environment and a two-drive regulator. It contains no pretrained model, no generative capability, and no data, and therefore poses no misuse risk requiring safeguards.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [N/A]

    Justification: No existing datasets, code packages, or models are used beyond standard open-source scientific libraries (`scipy`, BSD-3-Clause; `matplotlib`, PSF-based license), used in their ordinary capacity. The conceptual sources underlying the environment, notably the Rasch model and the Zone of Proximal Development, are cited in the references.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

Justification: The new asset is the simulation notebook, provided anonymized in the supplementary material. It is documented inline with markdown cells covering the research question, the preregistered predictions, the rationale for each design change, the quality gate criteria, and the prediction verdicts, and it emits a structured JSON of all results.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects. The "student" is a simulated agent in a synthetic environment; no data from real learners is collected or used.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects research was conducted, so no IRB approval or equivalent review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: We disclose LLM usage for transparency although it does not constitute a core-method component. No language model participates in the agent, the environment, the baselines, or the analysis: all are explicit numerical simulations. LLM assistance was confined to (i) code assistance, using Devin as an IDE with Kimi 3 and DeepSeek v4 Pro, and Claude Opus 5 and Claude Fable 5, and (ii) writing and editing assistance, using Claude Opus 5 and Claude Fable 5. All experimental design decisions, preregistered predictions, statistical procedures, and interpretations are the authors' own, and every reported number was produced by executing the released notebook.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.